# Project Proposal: Academic Citation Network Analyzer

**Course:** CSE412 Big Data & Large-Scale Computing
**Track:** C (Graph Analytics): HDFS → Spark → Spark GraphX
**Team:** *Member 1 · Member 2 · Member 3*

## 1. Problem statement

Citation counts are the default measure of a paper's importance, but they treat every citation as equal. A citation from a landmark paper and a citation from an obscure one count the same, and counts say nothing about how research fields connect. We want to answer three questions that need the **structure** of the citation network, not just per-paper counts:

1. **Influence:** Which papers are most influential when influence is propagated through the network (PageRank), and where does that ranking disagree with raw citation counts ("hidden gems")?
2. **Structure:** Do the research communities that emerge from citation links alone match the subject areas authors chose for their papers? How much do fields cite each other?
3. **Reach:** How far does the influence of the most important papers spread: how many papers build on them within one, two or three citation hops?

## 2. Dataset

**ogbn-arxiv** from the Open Graph Benchmark (Stanford SNAP), derived from the Microsoft Academic Graph:

- 169,343 arXiv computer-science papers (1971–2020), each with a publication year and one of 40 subject areas
- 1,166,243 directed citation edges
- Paper titles for every node from the companion `titleabs.tsv` (179,719 rows, ~200 MB uncompressed)
- Source: `https://snap.stanford.edu/ogb/data/nodeproppred/arxiv.zip` and `https://snap.stanford.edu/ogb/data/misc/ogbn_arxiv/titleabs.tsv.gz`

The raw release is spread over six files in four different layouts (positional value-per-line files, headed CSVs, a tab-separated title file inside a tar archive), so assembling a clean graph is real ETL work.

## 3. Why a single tool is not enough

- **Iterative graph algorithms.** PageRank, connected components and label propagation each need tens of passes over all 1.17 M edges, exchanging messages along edges on every pass. In SQL or MapReduce each pass is a separate self-join job. GraphX's Pregel model keeps the graph partitioned in memory and ships only messages between supersteps.
- **Graph traversal from many sources at once.** Multi-source shortest paths over the whole graph is a per-edge message-passing problem that tabular tools cannot express in one query.
- **Storage that outlives each job.** Each stage reads its input from and writes its output to HDFS, so stages can be re-run independently and data survives across Spark sessions. The same layout scales from this laptop to a multi-node cluster unchanged.

## 4. Tool combination and justification

| Stage | Tool | Why this tool |
|---|---|---|
| Storage | **HDFS** | Durable, block-distributed storage. Raw files are stored decompressed so they are splittable across parallel Spark tasks. Every hand-off between stages goes through HDFS |
| Stage 1: ETL | **Apache Spark** (core + SQL) | Joins five raw sources, handles the positional files with `zipWithIndex`, drops malformed / self / duplicate / dangling edges, infers citation direction, writes |

| Stage | Tool | Why this tool |
|---|---|---|
| | | columnar Parquet |
| Stage 2: Graph analytics | **Spark GraphX** | Property graph plus Pregel API: PageRank, weakly and strongly connected components, label propagation, triangle counting, shortest paths, triplet views |

## 5. Planned analysis and validation

| Question | Method | How we check the result means something |
|---|---|---|
| Influence | Dynamic PageRank | Spearman correlation with citation counts and top-k overlap; inspect where the two disagree |
| Communities | Label propagation (LPA) | NMI, ARI and purity against arXiv subject labels, compared with a random baseline of the same community sizes |
| Field structure | Triplet view → field-to-field citation matrix | Share of citations that stay within a field |
| Connectivity | Weakly / strongly connected components | Size of the giant component; citation cycles (should be rare in a time-ordered graph) |
| Local clustering | Triangle count → clustering coefficient | Compared with the clustering expected in a random graph of equal density |
| Reach | Multi-source shortest paths from the top PageRank papers | Cumulative papers reached per hop |

## 6. Expected outcomes

- A reproducible one-command pipeline (`run_pipeline.sh`) that moves the data HDFS → Spark → HDFS → GraphX → HDFS.
- Ranked lists of the most influential papers, hidden gems and rising stars, with titles.
- A quantitative answer to whether citation communities match human-assigned subject areas.
- A field-to-field citation map, plus charts and tables for the final report and demo.

## 7. Timeline

| Week | Milestone |
|---|---|
| 1 | Hadoop/HDFS running; Spark installed; dataset downloaded and ingested into HDFS (**check-in 1: working ingestion**) |
| 2 | Spark ETL stage with data-quality report; clean Parquet graph in HDFS |
| 3 | GraphX stage: PageRank, components, LPA; first results (**check-in 2: draft analytical results**) |
| 4 | Validation metrics, triangles, shortest paths, charts; report, README and demo video |

## 8. Risks and mitigation

| Risk | Mitigation |
|---|---|
| Iterative GraphX jobs exhaust driver memory on a laptop | Pregel checkpointing to HDFS; configurable driver memory; SCC can be skipped |
| Slow dataset mirror | Parallel ranged downloads with resume |

| Risk | Mitigation |
| --- | --- |
| Java version conflicts (course shell uses Java 8, Spark 4 needs 17+) | Scripts pin Java 21 explicitly |